

Manuel de Référence PureBasic OOP (Français)

Bienvenue dans la documentation officielle du transpileur et de la couche Orientée Objet pour PureBasic.

1. Introduction

PureBasic OOP apporte une syntaxe Orientée Objet de haut niveau tout en conservant la vitesse, la légèreté et la puissance de c

Caractéristiques Principales :

- **Classes & Méthodes** : Déclaration propre via ``Class`` et ``Method``.
- **Héritage Simple** : Support de l'héritage de classes via le mot-clé ``Extends``.
- **Polymorphisme VTable Dynamique** : Résolution automatique des tables virtuelles et des surcharges de méthodes (`*method o`
- **Appel Parent ``Super::``** : Possibilité d'appeler l'implémentation de la méthode parente via ``Super::Methode(...)``.
- **Encapsulation** : Gestion de la visibilité des attributs et méthodes (``Public``, ``Protected``, ``Private``).
- **Constructeurs et Destructeurs** : Instanciation automatique (``New Classe(...)``) et libération mémoire propre (``*obj\Free()``).

2. Syntaxe & Grammaire Objet

2.1 Déclaration de Classes et Héritage

```
`` `oop
; Classe de base
Class Animal
    Protected nom.s
    Protected age.i

    Public Method Init(nom_p.s, age_p.i)
    Public Method Crier()
    Public Method Free()
EndClass

; Classe dérivée
Class Chien Extends Animal
    Protected race.s

    Public Method Init(nom_p.s, age_p.i, race_p.s)
    Public Method Crier()           ; Surcharge (Override)
    Public Method Rapporter(objet.s) ; Nouvelle méthode
    Public Method Free()           ; Destructeur
EndClass
`` `
```

2.2 Implémentation des Méthodes et ``Super::``

Chaque méthode accède à ses attributs internes via la référence d'instance ``This``.

Pour appeler la méthode parente, utilisez le préfixe ``Super::``.

```
`` `oop
Method Animal::Init(nom_p.s, age_p.i)
    This\nom = nom_p
    This\age = age_p
EndMethod

Method Animal::Crier()
    PrintN("[Animal] " + This\nom + " émet un son générique.")
EndMethod
```

```
Method Animal::Free()
    FreeStructure(This)
EndMethod

Method Chien::Init(nom_p.s, age_p.i, race_p.s)
    Super::Init(nom_p, age_p) ; Initialisation du parent
    This\race = race_p
EndMethod

Method Chien::Crier()
    Super::Crier() ; Appel du comportement parent
    PrintN("    ==> CHIEN (" + This\race + ") : Wouaf ! Wouaf !")
EndMethod

Method Chien::Rapporter(objet.s)
    PrintN(This\nom + " rapporte : " + objet)
EndMethod

Method Chien::Free()
    Super::Free()
EndMethod
```
```

## 2.3 Utilisation et Polymorphisme

```
```oop
; 1. Instanciatiion concrète
Define *monChien.Chien = New Chien("Médor", 4, "Labrador")
*monChien\Crier()
*monChien\Rapporter("la balle")

; 2. Polymorphisme dynamique via liste ou pointeur de type parent
NewList *refuge.Animal()
AddElement(*refuge()) : *refuge() = *monChien

ForEach *refuge()
    *refuge()\Crier() ; Dispatch dynamique vers Chien::Crier() !
    *refuge()\Free() ; Libération polymorphe
Next
```
```

---

## 3. Plomberie PureBasic Générée

Le transpileur convertit automatiquement les classes en constructions natives PureBasic :

1. `Interface Chien\_vt Extends Animal\_vt` contenant uniquement les nouvelles méthodes ajoutées.
2. `Structure Chien\_Inst Extends Animal\_Inst` avec le pointeur `\*VTable` en tête de structure.
3. `DataSection` contenant les adresses des procédures avec les méthodes surchargées correctement positionnées.
4. Fonctions constructeurs `New\_Chien(...)` allouant la structure et initialisant la `VTable`.

---

## 4. Guide d'Exécution & Compilation

Transpiler un fichier `.pbo` vers `.pb` :

```
```cmd
"compiler/transpiler.exe" "src/mon_fichier.pbo" "src/mon_fichier_generated.pb"
```
```

Compiler le code PureBasic généré :

```
``cmd
"C:\Program Files\PureBasic\Compilers\pbcompiler.exe" "src/mon_fichier_generated.pb" /CONSOLE /EXE "src/mon_fichier.exe"
``
```